**The Chinese University of Hong Kong, Shenzhen**
**School of Data Science**



**DDA5001 · Homework 2**
Due: 23:59, Oct 30.

**Instructions:**

- Homework problems must be clearly answered to receive full credit. Complete sentences that establish a clear logical progress are highly recommended.
- Submit one **PDF** with your answers and your **.py** code through Blackboard, combined in **last name-first name-hw2.zip**; e.g., **San-Zhang-hw2.zip**. Keep `p2/` and `p3/` separate so their same-named files do not overwrite each other. Include small verification scripts, but no datasets, environments, caches, or large trial archives.
- You may submit your homework multiple times before the deadline. Only your **latest submission will be graded**.
- The homework must be written in English.
- Late submission will not be graded.
- Discussion with peers is allowed, but do not copy another student's solution or a ready-made solution. Coding-agent assistance is **required in Problem 2(b)**. Disclose the actual agent interaction requested in that part. Check any generated code, and write the mathematical derivations and explanations in your own words. You are responsible for understanding and verifying all submitted work.

---

**Problem 1 (15pts). Concepts and Fundamental Knowledge**

(a) **(3pts)** For binary linear classifiers on $d$ input features, i.e., $d$-dimensional linear classifier, what is the VC dimension? Here $d$ does not count the appended constant feature.

(b) **(4pts)** For a fixed learned classifier, briefly state the difference between test error and out-of-sample error.

(c) **(4pts)** Consider minimizing a differentiable convex loss $\mathcal{L}(\boldsymbol{\theta})$ over $\mathbb{R}^d$ using gradient descent (GD). Which one of the following statements is **false**? Briefly explain each choice.

1) If $\nabla\mathcal{L}(\boldsymbol{\theta}) = \mathbf{0}$, then $\boldsymbol{\theta}$ is a global minimizer.

2) Convexity guarantees that GD never increases the loss, regardless of how large the positive learning rate is.

3) If the gradient is nonzero, a sufficiently small step in the negative-gradient direction decreases the loss.

4) A small change between two consecutive GD iterates does not, by itself, guarantee that the current iterate is close to a minimizer.

(d) **(4pts)** For a fixed input $\boldsymbol{x}$, multi-class logistic regression applies softmax to the class scores $\boldsymbol{\theta}_1^\top \boldsymbol{x}, \ldots, \boldsymbol{\theta}_K^\top \boldsymbol{x}$. The predicted class is the one with the largest probability. Which one of the following statements is **false**? Briefly explain each choice.

1) The class with the largest score also has the largest predicted probability.

2) Adding the same constant to all class scores leaves the predicted probabilities unchanged.

3) Multiplying all class scores by any positive constant leaves the predicted probabilities unchanged.

4) With a one-hot label, the cross-entropy loss for this sample is determined by the probability assigned to its true class.

**Problem 2 (40pts). Multi-class Logistic Regression and Optimization Methods**

In this problem, we will explore the performance of multi-class logistic regression under different optimization methods and different datasets.

(a) **(12pts)** As in the lecture, let $\boldsymbol{\Theta} = [\boldsymbol{\theta}_1, \ldots, \boldsymbol{\theta}_K] \in \mathbb{R}^{d \times K}$, where each column $\boldsymbol{\theta}_k$ corresponds to one class. For sample $(\boldsymbol{x}_i, y_i)$, let $\mathbf{1}_i \in \mathbb{R}^K$ be its one-hot label vector and let

$$\boldsymbol{q}_i = \text{softmax}(\boldsymbol{\Theta}^\top \boldsymbol{x}_i) \in \mathbb{R}^K$$

be the predicted class probabilities. The learning problem is

$$\begin{aligned} \min_{\boldsymbol{\Theta} \in \mathbb{R}^{d \times K}} \ \mathcal{L}(\boldsymbol{\Theta}) &:= -\frac{1}{n} \sum_{i=1}^{n} \mathbf{1}_i^\top \log\Big(\text{softmax}(\boldsymbol{\Theta}^\top \boldsymbol{x}_i)\Big) \\ &= \frac{1}{n} \sum_{i=1}^{n} H(\mathbf{1}_i, \boldsymbol{q}_i). \end{aligned} \tag{1}$$

Here $H$ denotes the cross-entropy loss for one sample:

$$H(\boldsymbol{p}, \boldsymbol{q}) := -\boldsymbol{p}^\top \log \boldsymbol{q} = -\sum_{j=1}^{K} p_j \log q_j,$$

where the natural logarithm is applied to each component of $\boldsymbol{q}$.

Derive $\nabla_{\boldsymbol{\Theta}} \mathcal{L}(\boldsymbol{\Theta})$ by following the steps below. First consider a **single sample** and omit the index $i$: $\boldsymbol{x} = \boldsymbol{x}_i$, $\boldsymbol{p} = \mathbf{1}_i$, $\boldsymbol{z} = \boldsymbol{\Theta}^\top \boldsymbol{x}$ (called **logits**), and $\boldsymbol{q} = \text{softmax}(\boldsymbol{z})$. Here $p_j$ is 1 for the true class and 0 otherwise, so $\sum_{j=1}^{K} p_j = 1$.

1) **Differentiate cross-entropy (2pts).** Treat $\boldsymbol{p}$ as fixed and find $\dfrac{\partial H(\boldsymbol{p}, \boldsymbol{q})}{\partial q_j}$.

2) **Gradient of the logits $\boldsymbol{z}$ (4pts).** You may use the following derivative without proving it:

$$\frac{\partial q_j}{\partial z_k} = \begin{cases} q_k(1 - q_k), & j = k, \\ -q_j q_k, & j \neq k. \end{cases}$$

Use

$$\frac{\partial H}{\partial z_k} = \sum_{j=1}^{K} \frac{\partial H}{\partial q_j} \frac{\partial q_j}{\partial z_k}$$

to obtain $\nabla_{\boldsymbol{z}} H$. **Hint:** Separate the term $j = k$ from the other terms, then use $\sum_j p_j = 1$.

3) **Gradient of the parameters $\boldsymbol{\Theta}$ (4pts).** Since $z_k = \boldsymbol{\theta}_k^\top \boldsymbol{x}$, each column $\boldsymbol{\theta}_k$ affects only $z_k$. Use $\nabla_{\boldsymbol{\theta}_k} z_k = \boldsymbol{x}$ to find $\nabla_{\boldsymbol{\theta}_k} H$, then place these $K$ column gradients together to obtain $\nabla_{\boldsymbol{\Theta}} H \in \mathbb{R}^{d \times K}$.

4) **Average over the dataset (2pts).** Restore the sample index $i$ and average the single-sample gradients to obtain $\nabla_{\boldsymbol{\Theta}} \mathcal{L}(\boldsymbol{\Theta})$.

(b) **(28pts)** (AI assistance is **required for this part**; you remain responsible for understanding and checking the code.)

**Implement, verify, and compare three optimizers with a coding agent.** Use your analytic gradient from part (a) to minimize the same mean cross-entropy objective (1) on two small datasets:

- **Digits**: 1,797 handwritten-digit images, each with $8 \times 8 = 64$ pixel features, and 10 classes. It is included in scikit-learn; no separate dataset download is needed.
- **Dry Bean**: 13,611 records, 16 numerical size/shape features, and 7 bean varieties. The framework automatically downloads and caches the official CSV (about 2.5 MB). These are extracted features, not raw images.

**The provided framework fixes the experiment.** It removes exact duplicates before splitting (68 for Dry Bean), uses a stratified 80%/20% training/test split with seed 666, and standardizes features using training-set statistics only. It appends a constant feature 1 to $\boldsymbol{x}_i$, so any intercept is included in $\boldsymbol{\Theta}$; there is no separate bias term $b$. Denote the parameter matrix for Digits by $\boldsymbol{\Theta}_{\text{Digits}}$, and that for Dry Bean by $\boldsymbol{\Theta}_{\text{Dry Bean}}$. Their dimensions are

$$\boldsymbol{\Theta}_{\text{Digits}} \in \mathbb{R}^{65 \times 10}, \qquad \boldsymbol{\Theta}_{\text{Dry Bean}} \in \mathbb{R}^{17 \times 7}.$$

Here $65 = 64 + 1$ and $17 = 16 + 1$ count the features, including the constant feature; 10 and 7 count the classes. Each row corresponds to one feature, and each column to one class. Each method starts independently from the same all-zero parameter matrix, with $\boldsymbol{\Theta}_0 = \boldsymbol{\Theta}_1 = \mathbf{0}$. Here $\boldsymbol{\Theta}_1$ is the starting matrix (0 completed updates), so $\boldsymbol{\Theta}_{T+1}$ is the matrix after $T$ updates. Use full-batch training without regularization, $T = 1000$ updates, $\mu = 0.05$ for both datasets, and $\beta = 0.90$ for momentum. Do not change the data processing, split, initialization, or settings in `code_source/p2/framework.py` for the required experiment.

Implement the three methods studied, for $k = 1, \ldots, T$:

$$\begin{aligned}
\text{GD:} \quad & \boldsymbol{\Theta}_{k+1} = \boldsymbol{\Theta}_k - \mu \nabla \mathcal{L}(\boldsymbol{\Theta}_k), \\
\text{Momentum:} \quad & \boldsymbol{\Theta}_{k+1} = \boldsymbol{\Theta}_k - \mu \nabla \mathcal{L}(\boldsymbol{\Theta}_k) + \beta(\boldsymbol{\Theta}_k - \boldsymbol{\Theta}_{k-1}), \\
\text{Nesterov:} \quad & \boldsymbol{W}_k = \boldsymbol{\Theta}_k + \frac{k-1}{k+2}(\boldsymbol{\Theta}_k - \boldsymbol{\Theta}_{k-1}), \qquad \boldsymbol{\Theta}_{k+1} = \boldsymbol{W}_k - \mu \nabla \mathcal{L}(\boldsymbol{W}_k).
\end{aligned}$$

Notice *where* each method evaluates its gradient. All three methods use one training-set gradient evaluation per update; no test data may enter the gradient. The iteration axis records completed updates, from 0 to 1000.

1) **Implementation (10pts).** Work with a coding agent to complete the six core functions in `code_source/p2/algorithms.py`: stable softmax, mean cross-entropy, your analytic matrix gradient, and the three updates. Give the agent your gradient from part (a), then check its implementation against that derivation. Keep the function interfaces and do not modify input arrays in place. Compute the loss stably from logits rather than taking the logarithm of an underflowed probability. Do not use a ready-made classifier or optimizer, or autodiff gradients for training. Autodiff may be used only as an independent check.

2) **Check the agent-generated code (6pts).** Before running the full experiments, use the following two small checks. You may use code for the calculations; you do not need to derive the gradient again or calculate every entry by hand.

   - **Check the gradient (2pts).** Use the four-sample example from `p2/test_p2.py`:

$$X = \begin{pmatrix} 1 & -1 & 1 \\ 2 & 0.5 & 1 \\ -1 & 2 & 1 \\ 0.5 & 0.2 & 1 \end{pmatrix}, \quad Y = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}, \quad \boldsymbol{\Theta} = \begin{pmatrix} 0.1 & -0.2 & 0.05 \\ 0.3 & -0.1 & 0.2 \\ 0.02 & 0.1 & -0.1 \end{pmatrix}.$$

     Each row of $X$ is one sample, and the corresponding row of $Y$ is its one-hot label. Use the mean cross-entropy loss from part (a). First compute the gradient using your formula. Then estimate each derivative numerically: change just that entry of $\boldsymbol{\Theta}$ by $+10^{-6}$ and $-10^{-6}$. Subtract the loss for the negative change from the loss for the positive change, and divide by $2 \times 10^{-6}$ (central finite differences). Report the two gradient matrices and the largest absolute difference between corresponding entries. Are they close to each other?

   - **Check one update of each accelerated method (4pts).** Each method carries **(2pts)**. Use the same $\boldsymbol{\Theta}$ above as the current parameter; obtain the previous parameter by subtracting 0.02 from every entry. Set $\mu = 0.05$, $\beta = 0.90$, and $k = 3$. For this check only, use the simple test gradient $G(A) = 2A$ for any matrix $A$, instead of the cross-entropy gradient. This lets you check the update code separately from the loss/gradient code. For momentum and Nesterov, report (i) the matrix at which your code evaluates $G$, and (ii) the parameter matrix after one update. Compare these outputs with the update formulas above, and explain in one or two sentences why the two gradient evaluation points differ.

   Run all the provided tests in `p2/test_p2.py` as well. Submit the numerical checks and your short explanations, not just a message saying that the tests passed.

3) **Compare how the three methods learn (6pts).** Run GD, momentum, and Nesterov on both datasets using the fixed settings above. Include the following in your report:

   - **Four figures (2pts).** For each dataset, include one cross-entropy figure and one accuracy figure. Each figure has six curves: the training and test curves for each of the three methods. The framework records updates $0, 5, \ldots, 1000$ and uses a log-scale axis for the loss figures.
   - **One results table (1pt).** For each method on each dataset, report the training/test cross-entropy and training/test accuracy after 1000 updates.
   - **Answer three questions using your figures and table (3pts).** Each question carries **(1pt)**.

(i) Which method reduces the *training loss* faster?

(ii) Which method has the lowest *test loss*, and which has the highest *test accuracy* at the end? Is the method with the lowest training loss also best on the test set?

(iii) Does any method's training loss keep decreasing while its test loss increases? Explain why this would suggest overfitting. If you do not observe this on a dataset, say so.

Discuss what you actually observe. You do not need to obtain a particular ranking or show overfitting on both datasets.

4) **AI collaboration case (6pts).** At most one page in total. Select one verification or implementation decision from this part. Include: (i) your initial testable expectation or concern; (ii) the agent/tool and model, if shown, together with your actual prompt and the relevant response excerpt; (iii) the numerical evidence you brought back to the agent and a focused follow-up exchange; and (iv) your final decision to revise or retain the code, supported by a formula, gradient check, update check, or experiment. Reference your verification/results instead of repeating all outputs. A correct first implementation can receive full credit: no invented bug, minimum number of chat turns, or full conversation dump is required. Retain the original conversation for possible review.

From the `code_source/` folder, install the packages in `requirements.txt`, then run:

```
python -m unittest discover -s p2 -v
python p2/run.py.
```

The tests initially fail until you complete the six functions. If the Dry Bean download is blocked, save the linked official CSV as `p2/data_cache/dry_bean.csv` and rerun; do not change the checksum or use a different version of the data.

**Problem 3 (45pts). Overfitting**

This problem studies overfitting by comparing two polynomial models on the same training data.

**Legendre polynomials.** The first two Legendre polynomials are $L_0(x) = 1$ and $L_1(x) = x$. For $k \geq 2$, they are defined recursively by

$$L_k(x) = \frac{2k-1}{k} x L_{k-1}(x) - \frac{k-1}{k} L_{k-2}(x).$$

Each $L_k$ is a polynomial of degree $k$. Their orthogonality on $[-1, 1]$ gives

$$\int_{-1}^{1} L_q(x) L_i(x)\, dx = \begin{cases} 0, & i \neq q, \\ \frac{2}{2q+1}, & i = q. \end{cases}$$

**Underlying target function.** For a target degree $Q_g$, draw independent coefficients $a_q \sim \mathcal{N}(0, 1)$ and define

$$g(x) = \frac{1}{C_Q} \sum_{q=0}^{Q_g} a_q L_q(x), \qquad C_Q = \sqrt{\sum_{q=0}^{Q_g} \frac{1}{2q+1}}.$$

The input $x$ is uniformly distributed on $[-1, 1]$. This normalization gives $\mathbb{E}_{a,x}[g(x)^2] = 1$, where the expectation is over both the random coefficients and the input. For a particular drawn target,

$$\mathbb{E}_x[g(x)^2] = \frac{1}{C_Q^2} \sum_{q=0}^{Q_g} \frac{a_q^2}{2q+1},$$

which need not equal 1. Once drawn, the same target is used for all samples in that trial.

**Generating the training dataset.** Draw $x_i$ independently and uniformly from $[-1, 1]$ and draw $\epsilon_i$ independently from $\mathcal{N}(0, 1)$, independently of the target coefficients. Set

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n, \qquad y_i = g(x_i) + \sigma\epsilon_i.$$

Thus $\sigma^2$ is the observation-noise variance.

**Problem setting.** We study the effects of the noise variance $\sigma^2$, the target degree $Q_g$, and the sample count $n$. A degree-$K$ model has the form

$$f_K(x) = \sum_{k=0}^{K} w_k L_k(x).$$

Its training error and out-of-sample error are

$$\mathrm{Er}_{\mathrm{in}}(f) = \frac{1}{n} \sum_{i=1}^{n} (f(x_i) - y_i)^2, \qquad \mathrm{Er}_{\mathrm{out}}(f) = \mathbb{E}_x[(f(x) - g(x))^2] = \frac{1}{2} \int_{-1}^{1} (f(x) - g(x))^2 \, dx.$$

Here $\mathrm{Er}_{\mathrm{out}}$ is evaluated against the **clean target** for a fixed target and fitted model, not against noisy test labels.

Fit $f_2$ and $f_{10}$ by minimizing their training errors on **exactly the same dataset**. Define the relative overfitting measure

$$\varrho_e = \mathrm{Er}_{\mathrm{out}}(f_{10}) - \mathrm{Er}_{\mathrm{out}}(f_2).$$

The degree-2 model is contained in the degree-10 model, so the optimal fits satisfy $\mathrm{Er}_{\mathrm{in}}(f_{10}) \leq \mathrm{Er}_{\mathrm{in}}(f_2)$. If $\varrho_e > 0$, the more flexible model has no larger training error but a larger out-of-sample error: this is evidence of overfitting **relative to** $f_2$. If $\varrho_e < 0$, $f_{10}$ generalizes better than $f_2$ in this comparison; it does **not** prove that $f_{10}$ cannot overfit relative to some other model.

(a) **Least-squares fit (8pts).** Suppose $n \geq K + 1$ and the sampled inputs are distinct. Write the data matrix and parameter/label vectors **(4pts)**, and derive the optimal coefficients for $K = 2$ and $K = 10$ **(4pts)**.

**Hint:** The column corresponding to degree $k$ contains $L_k(x_i)$, including the constant column $L_0(x_i) = 1$. In code, you may use `numpy.linalg.lstsq` rather than explicitly inverting a matrix.

(b) **Analytic out-of-sample error (10pts).** Derive a formula for $\mathrm{Er}_{\mathrm{out}}(f_K)$ and apply it to $f_2$ and $f_{10}$. The coefficient difference, including missing terms and target normalization, carries **(4pts)**; the orthogonality calculation with the correct uniform density carries **(6pts)**.

**Hint:** Expand $f_K - g$ in the Legendre basis, treating missing coefficients as zero, and use orthogonality. Remember the factor $1/C_Q$ in the target and the density $1/2$ of the uniform input distribution.

(c) **Implementation, experiments, and interpretation (27pts).** Complete `fit_polynomial` and `population_error` in `code_source/p3/algorithms.py`, using (a) and (b). Coding-agent assistance is **optional**; no interaction record is required for this part. Data generation, Legendre evaluation, loops, and plotting are provided. Target coefficients passed to `population_error` are **already divided by** $C_Q$ and are never passed to `fit_polynomial`.

**Implementation and verification (13pts).** The least-squares fitting function carries **(5pts)**, the analytic error function **(5pts)**, and numerical verification **(3pts)**: one check for fitting **(1pt)**, one for error **(1pt)**, and explanations of both checks **(1pt)**. Include the expected and computed values. For example, check recovery of a known noiseless polynomial and compare the analytic error with numerical integration. The supplied tests include such examples. Passing tests alone does not replace reporting the verification evidence.

Each column of a batched label array is a separate target: fit it independently on the shared inputs. Required shapes are in the function docstrings.

The framework fixes the random seed to **666** and uses **1000 independent trials for each grid cell**. For each cell, report the arithmetic mean

$$\bar{\varrho}_e = \frac{1}{1000} \sum_{r=1}^{1000} \left( \mathrm{Er}_{\mathrm{out}}(f_{10}^{(r)}) - \mathrm{Er}_{\mathrm{out}}(f_2^{(r)}) \right).$$

Trials are independent within each cell. Across cells, reused random numbers and prefixes of inputs and coefficients make comparisons paired and less noisy. Both models fit the same labels. Do not change the fixed settings, add regularization, or discard trials.

1) **Target-complexity heat map (4pts).** Fix $\sigma^2 = 0.1$, vary $Q_g \in \{1, 2, \ldots, 50\}$ and $n \in \{20, 25, \ldots, 120\}$, and produce the heat map of $\bar{\varrho}_e$ with horizontal axis $n$ and vertical axis $Q_g$.
2) **Noise-variance heat map (4pts).** Fix $Q_g = 20$, vary $\sigma^2 \in \{0, 0.05, \ldots, 2\}$ and $n \in \{20, 25, \ldots, 120\}$, and produce the heat map with horizontal axis $n$ and vertical axis $\sigma^2$.
3) **Interpretation (6pts).** Explain the colors and zero-mean contour **(2pts)**, the effects of sample count and noise variance **(2pts)**, and the role of target complexity, including the noiseless case **(2pts)**. Is increasing $Q_g$ always harmful? Can $\varrho_e > 0$ occur when $\sigma^2 = 0$? The contour is an empirical comparison, not proof of a universal sharp transition.

Run `python -m unittest discover -s p3 -v` and then `python p3/run.py` from the student code folder. The runner saves the two heat maps, raw numerical results, and an experiment manifest. Colors saturate at $\pm 0.25$ for visibility only; the underlying errors and arithmetic means are not clipped or smoothed.

Submit `p3/algorithms.py`, any small verification script, both heat maps, your numerical checks, and your discussion. Part (c) totals **(27pts)** $(13+4+4+6)$; Problem 3 totals **(45pts)** $(8 + 10 + 27)$.